

When to Review: Spaced Repetition for Continual Pre-Training of Language Models

Alankar Atreya¹ Devesh Batra¹ Yoages Kumar Mantri¹ Jeremy Bantug¹
Greig A Cowan¹ Raad Khraishi^{1,2}

¹NatWest AI Research ²University College London

Abstract

Continual pre-training of large language models must acquire new information without erasing old knowledge. Existing replay methods often choose a global old/new mixture and sample uniformly, ignoring that examples differ in how quickly they are forgotten. We formulate continual pre-training as adaptive review scheduling: the training loop should decide not only how much history to replay, but which examples should return at each step. We introduce Spaced Repetition Training (SRT), a continual learning framework inspired by cognitive science, which schedules sample-rehearsal using the SuperMemo-2 (SM-2) algorithm. SRT maintains per-example review state, maps per-example perplexity to a recall-quality signal, and schedules historical examples for retention and new examples for consolidation while leaving the model, objective, and optimizer unchanged. On temporally separated Wikipedia and code corpora, SRT improves the stability-plasticity trade-off, recovering 5 to 37 percentage points of old-knowledge accuracy lost by naive continual pre-training across model scales while preserving or improving new-knowledge acquisition. At larger scale, SRT preserves broad benchmark performance that naive continual pre-training and uniform replay substantially degrade. Experiments with vision and tabular data further suggest that the scheduling principle extends beyond language when paired with an appropriate recall signal.

1 Introduction

Large language models are increasingly updated after pre-training as facts change, software ecosystems evolve, and new domain corpora appear (Gururangan et al., 2020; Ibrahim et al., 2024). Continual pre-training (CPT) is cheaper than training from scratch, but updates on new data can erase previously learned knowledge: catastrophic forgetting (McCloskey and Cohen, 1989; Kirkpatrick et al., 2017). In LLMs, this forgetting can also degrade

broad capabilities such as multitask reasoning and factual recall (Li and Lee, 2024; Ibrahim et al., 2024).

Replay is an attractive mitigation strategy because it is architecture-agnostic and scales naturally to LLM training (Rolnick et al., 2019; Chen et al., 2025; Li et al., 2025). Most replay methods, however, operate at the mixture level: they choose how much historical data to include and then sample old examples uniformly or by a fixed curriculum. This ignores heterogeneity in retention difficulty. Some examples remain stable after one review, while others drift quickly under interference from new learning. Uniform replay can therefore spend budget on already-retained examples while missing fragile ones.

We instead treat CPT as an adaptive review-scheduling problem. Drawing on spaced repetition and difficulty-adaptive memory consolidation (Ebbinghaus, 1885; Cepeda et al., 2006), we ask which examples should be reviewed at the current training step, not only how much old data should be mixed in. Algorithms such as SuperMemo-2 already operationalize this idea for human memory through per-item review state (Wozniak, 1998); we adapt the same principle to language model training.

We introduce **Spaced Repetition Training (SRT)**, a drop-in scheduler for continual pre-training. Each replayable example maintains an ease factor, review count, interval, and due step. At review time, per-example perplexity is converted into a recall-quality score that updates the next interval: difficult examples return sooner, while confidently retained examples receive longer gaps. The scheduler applies symmetrically to historical examples for retention and new examples for consolidation, and it leaves the model architecture, objective, and optimizer unchanged.

Our contributions are as follows:

- We formulate replay-based LLM continual pre-training as adaptive review scheduling, motivated by spaced repetition and heterogeneous retention difficulty.
- We propose SRT, a sample-level SuperMemo-2 scheduler that converts per-example perplexity into recall quality and schedules both old and new examples without changing the model, loss, or optimizer.
- We show that SRT mitigates catastrophic forgetting more effectively than naive CPT and uniform replay while maintaining new-knowledge acquisition on temporally separated Wikipedia and code corpora, and that it preserves broad benchmark performance that both baselines degrade. Additional vision and tabular experiments suggest that the scheduling principle generalises beyond language.

2 Related Work

Catastrophic forgetting. Sequential training can overwrite representations needed for earlier tasks (McCloskey and Cohen, 1989). Standard mitigation strategies include regularization, which protects important parameters (Kirkpatrick et al., 2017; Zenke et al., 2017); architectural isolation or expansion (Rusu et al., 2016; Hung et al., 2019); and replay, which interleaves previous examples with new data (Rolnick et al., 2019; Parisi et al., 2019). For LLM CPT, replay is especially practical because task boundaries are weak, parameter growth is undesirable, and gradient-level regularization is expensive. SRT remains within the replay family but shifts the design question from mixture size to per-example review timing.

Continual pre-training of LLMs. Domain-adaptive pre-training can improve target-domain performance while shifting models away from prior capabilities (Gururangan et al., 2020). Recent CPT work studies learning-rate rewarming and re-decay, data mixtures, replay ratios, curricula, and temporal benchmarks (Ibrahim et al., 2024; Chen et al., 2025; Li et al., 2025). Other studies show that continual updates can degrade established benchmarks such as multitask language understanding and factual recall (Li and Lee, 2024). These methods primarily tune optimization or mixture-level replay. SRT is complementary: given a replay budget, it decides which stored examples should be reviewed now.

Human-inspired replay. SuperMemo-2 maps recall quality to expanding inter-review intervals through per-item state (Wozniak, 1998). Recent neural methods import related ideas via self-synthesized rehearsal, Leitner-style queues, or forgetting-curve replay (Huang et al., 2024; M’hamdi and May, 2024; Feng et al., 2026). SRT differs by maintaining SuperMemo-2 state for individual training examples and deriving recall quality directly from per-example perplexity computed in the standard forward pass, requiring no synthesis pipeline or held-out probe set.

3 Method

3.1 Problem Setup

Let p_θ denote a language model with parameters θ initialized from a pre-trained checkpoint. We consider a continual pre-training setting in which the model is updated on an incoming corpus $\mathcal{D}_{\text{new}} = \{x_j^{\text{new}}\}$ while also retaining access to a historical corpus $\mathcal{D}_{\text{old}} = \{x_i^{\text{old}}\}$ representing knowledge acquired before the update. The goal of continual pre-training is not simply to minimize next-token loss on $\mathcal{D}_{\text{new}}$, but to acquire new information while preserving performance on $\mathcal{D}_{\text{old}}$. We express this as a weighted combination of evaluation risks,

$$\min_{\theta'} \lambda \mathcal{R}_{\text{old}}(\theta') + (1 - \lambda) \mathcal{R}_{\text{new}}(\theta'), \quad (1)$$

where $\mathcal{R}_{\text{old}}$ and $\mathcal{R}_{\text{new}}$ are evaluation risks on old and new data respectively, and $\lambda \in [0, 1]$ controls the stability-plasticity trade-off.

In practice, training operates under a finite budget of optimization steps and tokens. At step t , a mini-batch $\mathcal{B}_t$ of size B must allocate exposure between old and new examples,

$$\mathcal{B}_t = \mathcal{B}_t^{\text{old}} \cup \mathcal{B}_t^{\text{new}}, \quad |\mathcal{B}_t^{\text{old}}| + |\mathcal{B}_t^{\text{new}}| \leq B. \quad (2)$$

Naive continual pre-training sets $|\mathcal{B}_t^{\text{old}}| = 0$ and spends the entire batch on $\mathcal{D}_{\text{new}}$. Replay-based methods instead allocate a fraction of each batch to examples drawn from $\mathcal{D}_{\text{old}}$. The central design question is therefore not only how much of each batch to allocate to old data, but which specific examples should be drawn from each pool at step t , given that retention difficulty varies across both corpora.

3.2 Spaced Repetition Training

SRT addresses this question by maintaining review state and scheduling using the SuperMemo-2 al-

gorithm (Wozniak, 1998). Each example $x_i \in \mathcal{D}_{\text{old}} \cup \mathcal{D}_{\text{new}}$ is associated with a review state

$$s_i(t) = (E_i(t), n_i(t), I_i(t), d_i(t)), \quad (3)$$

where E_i is the ease factor controlling interval growth, n_i is the number of consecutive successful reviews, I_i is the current inter-review interval measured in training steps, and d_i is the next due step. Following standard SuperMemo-2 initialization, we set $E_i = 2.5$, $n_i = 0$, and $I_i = 1$, with initial due times staggered uniformly over the first few training steps to avoid reviewing all items simultaneously. State is maintained symmetrically for examples in $\mathcal{D}_{\text{old}}$ and $\mathcal{D}_{\text{new}}$, so SRT applies the same scheduling logic to historical examples (where it determines replay timing) and to incoming examples (where it determines consolidation timing).

Due-set sampling. At training step t , we define the sets of examples in $\mathcal{D}_{\text{old}}$ and $\mathcal{D}_{\text{new}}$ whose next review is due,

$$\mathcal{M}_t^{\text{old}} = \{x_i \in \mathcal{D}_{\text{old}} : d_i \leq t\}, \quad (4)$$

$$\mathcal{M}_t^{\text{new}} = \{x_j \in \mathcal{D}_{\text{new}} : d_j \leq t\}. \quad (5)$$

The training batch at step t allocates $\lfloor \rho B \rfloor$ slots to due old examples sampled from $\mathcal{M}_t^{\text{old}}$ and the remaining slots to due new examples sampled from $\mathcal{M}_t^{\text{new}}$, where $\rho \in [0, 1]$ is the target old-exposure fraction. Both old and new examples are therefore selected by the scheduler, not only those drawn from $\mathcal{D}_{\text{old}}$. If either due set is empty at step t , the unused budget is released to the other stream, ensuring that scheduling never blocks training progress.

Perplexity-derived recall quality. When a selected example $x_i = (x_{i,1}, \dots, x_{i,T_i})$ is reviewed, we compute its token-average negative log-likelihood under the current model,

$$L_i(t) = -\frac{1}{T_i} \sum_{k=1}^{T_i} \log p_{\theta_t}(x_{i,k} \mid x_{i,<k}), \quad (6)$$

and its perplexity $\text{PPL}_i(t) = \exp(L_i(t))$. Perplexity is computed during the standard forward pass and requires no auxiliary evaluation. We convert perplexity into the discrete recall quality score $q_i(t) \in \{1, 2, 3, 4, 5\}$ used by SuperMemo-2 via monotonically increasing thresholds $0 < \tau_5 < \tau_4 < \tau_3 < \tau_2 < \tau_1$ that partition model perfor-

mance into quality levels,

$$q_i(t) = \max\left(0, 5 - \sum_{k=1}^5 \mathbf{1}[\text{PPL}_i(t) \geq \tau_k]\right). \quad (7)$$

Lower perplexity yields higher recall quality, with $q = 5$ corresponding to fluent retention and $q = 0$ to essentially no model on the example. Threshold values were selected empirically; Section 6 analyses sensitivity to their scaling.

The recall quality $q_i(t)$ is computed from the training-time loss at the current parameters θ_t , that is, from the same forward pass used to obtain the gradient, before the optimizer step that produces θ_{t+1} . It therefore reflects how well the model retained x_i prior to the current update, not how well it fits x_i after being trained on it.

For non-generative models where perplexity is not defined, the same scheduler can be applied by replacing $\text{PPL}_i(t)$ with an alternative confidence-based signal. Details of this adaptation are provided in Appendix C.2.

Review state update. Given recall quality q_i , the ease factor is updated according to the SuperMemo-2 rule,

$$\Delta(q_i) = 0.1 - (5 - q_i)(0.08 + 0.02(5 - q_i)), \quad (8)$$

$$E_i \leftarrow \max(1.3, E_i + \Delta(q_i)). \quad (9)$$

If $q_i < 3$, the example is treated as forgotten and we reset $n_i \leftarrow 0$ and $I_i \leftarrow 1$, scheduling immediate re-review. Otherwise we increment n_i and update the interval according to

$$I_i \leftarrow \begin{cases} 1, & n_i = 1, \\ 6, & n_i = 2, \\ \lceil I_i E_i \rceil, & n_i > 2. \end{cases} \quad (10)$$

The next review is scheduled by setting $d_i \leftarrow t + I_i$. The model parameters θ are updated using standard next-token cross-entropy loss on $\mathcal{B}_t$, identical to the underlying language modelling objective. SRT therefore introduces no additional loss terms, no architectural changes, and no learned parameters beyond those already present in p_θ . The full procedure is summarized in Algorithm 1.

4 Experimental Setup

4.1 Corpora

We construct temporally grounded corpora in two domains where knowledge evolves measurably

Algorithm 1 One SRT training step.

Require: corpora $\mathcal{D}_{\text{old}}, \mathcal{D}_{\text{new}}$, old-exposure cap ρ , batch size B , step t

- 1: $\mathcal{M}_t^{\text{old}} \leftarrow \{x_i \in \mathcal{D}_{\text{old}} : d_i \leq t\}$ $\triangleright$ due old examples
- 2: $\mathcal{M}_t^{\text{new}} \leftarrow \{x_j \in \mathcal{D}_{\text{new}} : d_j \leq t\}$ $\triangleright$ due new examples
- 3: $b^{\text{old}} \leftarrow \min(|\mathcal{M}_t^{\text{old}}|, \lfloor \rho B \rfloor)$
- 4: sample $\mathcal{B}_t^{\text{old}} \subseteq \mathcal{M}_t^{\text{old}}$ with $|\mathcal{B}_t^{\text{old}}| = b^{\text{old}}$
- 5: sample $\mathcal{B}_t^{\text{new}} \subseteq \mathcal{M}_t^{\text{new}}$ to fill remaining slots
- 6: $\mathcal{B}_t \leftarrow \mathcal{B}_t^{\text{old}} \cup \mathcal{B}_t^{\text{new}}$
- 7: **for** $x_i \in \mathcal{B}_t$ **do** $\triangleright$ scored at θ_t , before the update
- 8: compute $\text{PPL}_i(t)$ via Eq. 6 and $q_i(t)$ via Eq. 7
- 9: **end for**
- 10: update θ on $\mathcal{B}_t$ using next-token loss $\triangleright \theta_t \rightarrow \theta_{t+1}$
- 11: **for** $x_i \in \mathcal{B}_t$ **do** $\triangleright$ update review state only
- 12: update E_i, n_i, I_i via Eqs. 9–10
- 13: set $d_i \leftarrow t + I_i$
- 14: **end for**

over time: encyclopedic text and source code. Both domains are split into an *old* corpus $\mathcal{D}_{\text{old}}$ representing information available before a temporal cutoff and a *new* corpus $\mathcal{D}_{\text{new}}$ representing post-cutoff information, enabling evaluation that distinguishes retained knowledge from acquired knowledge.

Wikipedia. For encyclopedic text, we align timestamped English Wikipedia snapshots by normalized article title, producing aligned old-new pairs corresponding to the same entity at different points in time. This formulation follows temporal knowledge benchmarks that evaluate whether language models can acquire updated information while retaining prior knowledge (Jang et al., 2022; Cheng et al., 2024; Li et al., 2025). The new-data corpus is extracted from sentence-level diffs between aligned pairs, capturing inserted spans and the updated side of modified spans, which preserves localized factual updates while filtering unchanged text. Full snapshot selection, topic stratification, and diff-extraction details are provided in Appendix A.1.

Code. For source code, we construct temporally separated corpora from GitHub repositories selected by creation and commit timestamps. Old examples are drawn from repositories that existed before January 2022, and new examples from repositories created after July 2024, with verification that $\mathcal{D}_{\text{new}}$ repositories did not exist before the cutoff window. Repositories span thirteen widely used programming languages. Unlike the Wikipedia corpus, old and new code examples are not aligned revisions of the same artifact; they represent temporally distinct samples from evolving software distributions, reflecting how software ecosystems shift through new libraries, repositories, and con-

ventions rather than incremental edits to existing artifacts. Full repository selection criteria and language coverage are provided in Appendix A.2.

Corpus statistics. Corpus sizes for both domains are summarized in Appendix A (Table 8). For Wikipedia, the old and new training corpora each contain 31,729 aligned examples. For code, the old corpus contains 22,055 examples and the new corpus contains 70,643 examples.

4.2 Models and Training

We evaluate SRT across two model scales. Smaller-scale experiments use TinyLlama-1.1B-Chat (Zhang et al., 2024), which provides a fast iteration platform for ablations and main comparisons. Larger-scale experiments use Llama-3.2-3B-Instruct (Grattafiori et al., 2024; Meta AI, 2024), a 3-billion-parameter instruction-tuned model released by Meta with a knowledge cutoff of December 2023. We continually pre-train from the instruction-tuned checkpoint rather than a base checkpoint because updating already-deployed instruct models on fresh data is the realistic practitioner scenario our method targets.

All conditions use the same continual pre-training recipe, following established practice for replay-based LLM continual pre-training (Ibrahim et al., 2024). We use causal language modeling with the next-token cross-entropy objective, AdamW optimization, linear learning-rate scheduling with warmup and decay, gradient clipping, and bfloat16 precision where supported. Hyperparameters were tuned on a held-out development split and held fixed across all baselines and SRT conditions to ensure that performance differences reflect the replay strategy rather than optimization differences. Complete hyperparameter values are provided in Appendix B.

4.3 Baselines

We compare four conditions against the original pre-trained checkpoint. **Base** is the pre-trained model before continual update. It establishes the starting point for both old-knowledge retention and new-knowledge acquisition. **CPT** is naive continual pre-training, where each training batch is drawn entirely from $\mathcal{D}_{\text{new}}$. This represents the standard no-replay baseline and exhibits the canonical forgetting failure mode. **Uniform Replay** draws each batch using a nominal 20/80 old/new exposure ratio, with old examples sampled uniformly at

random from $\mathcal{D}_{\text{old}}$ and new examples sampled uniformly at random from $\mathcal{D}_{\text{new}}$. This baseline shares the same nominal replay budget and pool structure as SRT but uses no per-example scheduling for either pool, isolating the contribution of adaptive review timing. **PPL-Prioritised** is a difficulty-aware replay baseline that isolates whether SRT’s gains come from spaced-repetition scheduling or simply from attending to hard examples. It uses the same 20/80 old/new exposure ratio as Uniform Replay and SRT, but within each pool it selects the highest-perplexity examples rather than sampling uniformly, without maintaining any SM-2 review state or interval scheduling. It therefore replays the currently hardest examples at every step. Because its role is to separate difficulty-prioritisation from interval-based scheduling rather than to serve as a scale-general baseline, we evaluate it on both TinyLlama settings (Wikipedia and code) as a diagnostic control. **SRT** uses the same nominal 20/80 old/new exposure cap and pool structure as Uniform Replay, but selects examples in each batch using SuperMemo-2 due times and perplexity-derived recall quality as described in Section 3.2. Both old and new examples are selected by the scheduler. The old-exposure fraction is set to $\rho = 0.2$, matching Uniform Replay so that the only difference between the two conditions is whether within-pool selection is scheduled or uniform.

4.4 Evaluation

We evaluate along two complementary axes. Source-grounded question answering measures retention and acquisition on the temporally split corpora. Standard capability benchmarks measure whether continual updates degrade broad model abilities beyond the updated domains.

Source-grounded QA. For each of the Wikipedia and code domains, we generate multiple-choice QA benchmarks in MMLU-style format directly from source passages in $\mathcal{D}_{\text{old}}$ and $\mathcal{D}_{\text{new}}$, producing matched old and new evaluation splits of 500 questions each. Questions are generated from source content and manually checked for answerability and label support. We report accuracy as the mean across questions together with the bootstrap standard deviation computed from 10,000 resamples.

Broad capability benchmarks. We evaluate on four widely used benchmarks: MMLU (Hendrycks et al., 2021) for multitask knowledge and rea-

soning, BBH (Suzgun et al., 2023) for hard reasoning tasks, GSM8K (Cobbe et al., 2021) for grade-school mathematical reasoning, and PIQA (Bisk et al., 2020) for physical commonsense. All broad capability benchmarks are evaluated using the EleutherAI LM Evaluation Harness (Gao et al., 2023), which provides standardized few-shot prompts, scoring procedures, and task implementations across models. Using a single evaluation framework ensures that benchmark scores are directly comparable across baselines and SRT, and that our reported numbers are reproducible by other researchers. These benchmarks are not optimization targets for any condition and serve to detect whether continual updates damage capabilities unrelated to the updated domains.

5 Results

We evaluate SRT along two axes: source-grounded temporal QA (Section 5.1) and broad capability benchmarks (Section 5.2).

5.1 Temporal QA: Retention and Acquisition

Table 1 reports source-grounded QA accuracy across both models. The pattern is consistent at both scales: naive CPT improves new-knowledge accuracy at the cost of retention, uniform replay partially mitigates the drop, and SRT recovers retention while preserving or improving new-knowledge accuracy.

Naive CPT degrades retention. On TinyLlama, CPT raises new accuracy from 13.1% to 17.0% on Wikipedia data but old accuracy collapses from 54.3% to 11.7%. The same pattern holds on Llama-3.2-3B-Instruct, where old Wikipedia accuracy drops from 50.2% to 43.0%.

Scheduling, not exposure, drives SRT’s gains. UNIFORM REPLAY reaches 25.2% and 46.2% old Wikipedia accuracy on TinyLlama and Llama-3.2-3B-Instruct respectively, in both cases substantially above CPT but below SRT (49.0% and 51.6%). Because UNIFORM REPLAY and SRT share the same 20/80 exposure cap and pool structure and differ only in within-pool selection, this gap is attributable to scheduling rather than exposure. The gap is larger at the smaller model scale (23.8 percentage points on TinyLlama versus 5.4 on Llama-3.2-3B-Instruct), suggesting that smaller models benefit more from prioritized review while larger models retain more knowledge under uniform replay alone.

Scheduling versus difficulty alone. To separate interval-based scheduling from simple difficulty-prioritisation, we compare SRT against a PPL-Prioritised baseline that replays the highest-perplexity old examples without SM-2 review state (Table 1). Difficulty-awareness alone is clearly beneficial: on Wikipedia, PPL-Prioritised improves old-knowledge retention over UNIFORM REPLAY (27.4% vs. 25.2%) and achieves the highest new-knowledge accuracy of any method (38.8%). However, the two difficulty-aware methods occupy different points on the stability-plasticity spectrum. PPL-Prioritised is markedly more plastic, acquiring new knowledge aggressively but recovering far less old knowledge than SRT (27.4% vs. 49.0% on Wikipedia, a 21.6-point retention gap). We attribute this to its greedy focus on the currently hardest examples, which neglects intermediate-difficulty examples that SRT’s interval scheduling continues to revisit along their forgetting trajectory. On code, SRT outperforms PPL-Prioritised on all three metrics. Overall, SRT achieves the best old-knowledge retention on both domains and the best combined score, while PPL-Prioritised attains stronger new-knowledge accuracy on Wikipedia; interval-based scheduling thus provides a substantially better retention-acquisition balance than one-shot difficulty ranking, though the two methods prioritise that balance differently.

SRT improves both retention and acquisition. SRT produces the best combined score on every model-domain pair in Table 1. On Wikipedia, it restores old accuracy to or above the base-model level at both scales while matching or exceeding CPT on new accuracy. On code QA, SRT is the only method that simultaneously improves both old and new accuracy relative to CPT on both models. UNIFORM REPLAY is competitive with CPT on code at both scales but trails SRT consistently, confirming that scheduling provides value across model scales even when uniform replay alone is adequate. To corroborate these findings on a benchmark we did not construct, we additionally evaluate on TemporalWiki (Jang et al., 2022), an external factual-probe set. SRT substantially limits the perplexity degradation that CPT incurs on this benchmark, consistent with the retention advantage measured on our source-grounded QA. Full results and caveats are provided in Appendix D.

5.2 Broad Capability Preservation

Table 2 reports accuracy on MMLU, BBH, GSM8K, and PIQA. On TinyLlama, all methods remain within standard-deviation bounds of the base model, reflecting that the base model’s MMLU and GSM8K scores are near random and leave little signal to disturb.

The Llama-3.2-3B-Instruct results reveal a stark pattern. CPT substantially degrades all four benchmarks, with the largest drop on GSM8K (77.6% to 38.8%): a 38.8-point decline despite no obvious relationship between the temporally localized training data and grade-school math. UNIFORM REPLAY performs even worse on reasoning, dropping BBH to 8.4% and GSM8K to 6.8%. Re-exposing the model to old data uniformly therefore damages reasoning capabilities more severely than no replay at all, suggesting that uniform replay introduces interference patterns that disrupt reasoning circuits rather than mitigating forgetting. SRT largely preserves base-model performance across all four benchmarks (57.8%, 52.6%, 76.7%, 74.3%), and as shown in Table 1, this preservation does not come at the cost of adaptation.

5.3 Auxiliary Non-Language Evidence

To test whether the scheduler is tied to language perplexity, we also summarize class-incremental vision and tabular experiments, with details in Appendix C. Replacing perplexity with predicted-class confidence gives the same qualitative pattern: on MNIST, Fashion-MNIST, CIFAR-10, and Wine, SRT obtains the highest overall accuracy (91.6%, 65.8%, 53.1%, and 53.3%, respectively). We treat these experiments as supporting evidence that adaptive review scheduling is modality-agnostic when paired with an appropriate recall signal; the central claim remains LLM continual pre-training.

5.4 Computational Overhead

SRT adds a forward pass per reviewed example beyond standard continual pre-training to compute the perplexity-based recall quality score, which introduces some computational cost. To quantify this overhead, we measure wall-clock time, per-step time, and throughput for CPT, UNIFORM REPLAY, and SRT under matched configurations on the same hardware. Table 3 reports the results.

The overhead of SRT relative to CPT is 14.7% in wall-clock time, corresponding to a 14.6% reduction in tokens per second. UNIFORM REPLAY

Model	Method	Wikipedia QA			Code QA		
		Old	New	Combined	Old	New	Combined
TinyLlama	BASE	54.3 $\pm$ 2.1	13.1 $\pm$ 1.5	33.7 $\pm$ 1.5	19.5 $\pm$ 1.8	11.6 $\pm$ 1.4	15.6 $\pm$ 1.1
	CPT	11.7 $\pm$ 1.4	17.0 $\pm$ 1.4	14.4 $\pm$ 1.1	17.4 $\pm$ 1.7	14.8 $\pm$ 1.6	16.6 $\pm$ 1.2
	UNIFORM REPLAY	25.2 $\pm$ 1.9	13.6 $\pm$ 1.5	19.4 $\pm$ 1.4	14.0 $\pm$ 1.6	15.4 $\pm$ 1.6	14.7 $\pm$ 1.1
	PPL-Prior.	27.4 $\pm$ 2.0	38.8 $\pm$ 2.2	33.1 $\pm$ 1.5	20.8 $\pm$ 3.6	13.6 $\pm$ 3.0	17.2 $\pm$ 2.3
	SRT	49.0 $\pm$ 2.1	20.0 $\pm$ 1.7	34.6 $\pm$ 1.5	24.4 $\pm$ 1.9	20.7 $\pm$ 1.8	22.6 $\pm$ 1.3
Llama-3.2-3B-Inst.	BASE	50.2 $\pm$ 2.2	39.8 $\pm$ 2.1	45.0 $\pm$ 1.5	55.0 $\pm$ 2.2	52.6 $\pm$ 2.2	53.8 $\pm$ 1.5
	CPT	43.0 $\pm$ 2.2	42.4 $\pm$ 2.2	42.7 $\pm$ 1.5	54.6 $\pm$ 2.2	53.2 $\pm$ 2.2	53.9 $\pm$ 1.5
	UNIFORM REPLAY	46.2 $\pm$ 2.2	39.8 $\pm$ 2.1	43.0 $\pm$ 1.5	56.0 $\pm$ 2.2	57.2 $\pm$ 2.2	56.6 $\pm$ 1.6
	SRT	51.6 $\pm$ 2.2	42.4 $\pm$ 2.2	47.0 $\pm$ 1.5	60.8 $\pm$ 2.2	57.8 $\pm$ 2.2	59.3 $\pm$ 1.5

Table 1: Source-grounded QA accuracy (mean $\pm$ bootstrap standard deviation, %). Each old/new split has 500 held-out questions. UNIFORM REPLAY and SRT share the same 20/80 exposure cap and pool structure, differing only in whether within-pool selection is uniform or scheduled.

Model	Method	MMLU	BBH	GSM8K	PIQA
TinyLlama	BASE	24.9 $\pm$ 3.6	27.1 $\pm$ 0.1	2.4 $\pm$ 0.0	73.4 $\pm$ 0.0
	CPT	24.9 $\pm$ 0.0	26.9 $\pm$ 0.0	2.0 $\pm$ 0.0	73.3 $\pm$ 1.0
	UNIFORM REPLAY	24.4 $\pm$ 0.0	26.8 $\pm$ 0.0	3.3 $\pm$ 0.0	73.2 $\pm$ 0.0
	SRT	24.4 $\pm$ 0.0	25.8 $\pm$ 0.0	2.5 $\pm$ 0.0	72.2 $\pm$ 0.0
Llama-3.2-3B-Inst.	BASE	57.5 $\pm$ 0.0	53.9 $\pm$ 0.0	77.6 $\pm$ 1.2	75.1 $\pm$ 1.0
	CPT	51.4 $\pm$ 0.4	44.3 $\pm$ 0.6	38.8 $\pm$ 1.3	69.0 $\pm$ 0.7
	UNIFORM REPLAY	50.0 $\pm$ 0.0	8.4 $\pm$ 0.0	6.8 $\pm$ 0.1	74.9 $\pm$ 1.0
	SRT	57.8 $\pm$ 0.1	52.6 $\pm$ 0.1	76.7 $\pm$ 1.2	74.3 $\pm$ 1.0

Table 2: Broad capability benchmark accuracy (mean $\pm$ standard deviation, %). At the Llama-3.2-3B-Instruct scale, both CPT and UNIFORM REPLAY degrade reasoning benchmarks (BBH, GSM8K) substantially; SRT preserves base-model performance.

also incurs overhead relative to CPT (10.8% wall-clock, 13.0% throughput reduction), reflecting the cost of sampling and combining old and new examples into each batch. The additional cost of SRT relative to UNIFORM REPLAY is therefore only 3.5% in wall-clock time, attributable to the recall quality computation and SuperMemo-2 state updates. Given the substantial retention gains reported in Section 5.1, this scheduling overhead is a favourable trade-off for practitioners who care about old-knowledge preservation.

6 Ablations

We conduct three ablation studies to evaluate the sensitivity of SRT to its main design choices: the replay budget, the perplexity-to-quality threshold scaling, and the initial ease factor. The replay-ratio ablation uses TinyLlama Wikipedia QA to probe sensitivity to old-new exposure allocation, while the threshold and ease-factor ablations use TinyLlama code QA as a representative domain.

Replay budget. Table 4 sweeps the old/new exposure ratio ρ on TinyLlama Wikipedia. The best configuration is 20/80, reaching 49.6% old and

20.0% new accuracy. Performance is sensitive to the budget at both extremes. Old-heavy schedules (50/50 and beyond) reduce exposure to new data and fail to recover old-knowledge accuracy because the model has less budget for both consolidation and adaptation. New-heavy schedules (5/95) behave closer to CPT and lose old knowledge for the same reason. The 20/80 split aligns with the matched-budget design used in the main comparison and provides enough old-data exposure for SRT’s scheduler to operate while preserving sufficient adaptation capacity.

Perplexity-to-quality threshold scaling. We test sensitivity to the threshold values used in Eq. 7. A scalar multiplier α is applied uniformly to all five thresholds ($\tau_1, \dots, \tau_5$), so $\alpha < 1$ produces stricter scoring (examples need lower perplexity to be considered retained) and $\alpha > 1$ produces relaxed scoring. Table 5 reports TinyLlama code QA accuracy across $\alpha \in \{0.5, 1.0, 1.5\}$. The default $\alpha = 1.0$ is best, reaching 22.6% combined accuracy. Both stricter ($\alpha = 0.5$) and more relaxed ($\alpha = 1.5$) thresholds degrade performance. The effect is asymmetric: relaxation hurts more than

Metric	CPT	UNIFORM REPLAY	SRT
Wall-clock time (s)	960.07	1063.87	1101.36
Time per step (s)	9.19	10.33	10.75
Tokens per second	455.83	396.51	380.90
Tokens per step	4096	4096	4096
Overhead vs. CPT	—	+10.8%	+14.7%
Overhead vs. UNIFORM REPLAY	—	—	+3.5%

Table 3: Computational overhead of UNIFORM REPLAY and SRT relative to CPT under matched configurations. SRT adds approximately 15% wall-clock overhead relative to no-replay training, of which most is shared with UNIFORM REPLAY due to batch construction cost; the additional scheduling-specific overhead is only 3.5%.

Old/New	Old	New	Combined
5/95	23.6 $\pm$ 1.8	13.3 $\pm$ 1.5	18.4 $\pm$ 1.3
20/80	49.6 $\pm$ 2.1	20.0 $\pm$ 1.7	34.6 $\pm$ 1.5
35/65	23.4 $\pm$ 1.9	15.4 $\pm$ 1.6	19.4 $\pm$ 1.2
50/50	21.8 $\pm$ 1.9	13.8 $\pm$ 1.5	17.8 $\pm$ 1.1
65/35	20.6 $\pm$ 1.8	12.2 $\pm$ 1.4	16.4 $\pm$ 1.2
80/20	23.5 $\pm$ 1.9	14.2 $\pm$ 1.6	18.8 $\pm$ 1.2
95/5	27.4 $\pm$ 1.9	19.7 $\pm$ 1.7	23.6 $\pm$ 1.3

Table 4: TinyLlama Wikipedia QA accuracy as a function of the SRT old/new exposure ratio. Accuracy is mean $\pm$ bootstrap standard deviation, %.

α	Code Old	Code New	Combined
0.5	13.2 $\pm$ 1.4	8.6 $\pm$ 1.2	10.9 $\pm$ 1.0
1.0	24.4 $\pm$ 1.9	20.7 $\pm$ 1.8	22.6 $\pm$ 1.3
1.5	4.8 $\pm$ 1.1	5.4 $\pm$ 1.0	5.1 $\pm$ 0.9

Table 5: Sensitivity to perplexity threshold scaling on TinyLlama code QA. The scalar multiplier α is applied uniformly to all five thresholds in Eq. 7.

equivalent strictness, because over-relaxed thresholds assign high quality to poorly retained examples and stop scheduling them for review.

Initial ease factor. We vary the initial ease factor E_0 , which controls how quickly review intervals expand after early successful reviews. Table 6 reports TinyLlama code QA across $E_0 \in \{1.5, 2.5, 3.5\}$. The default SM-2 value $E_0 = 2.5$ is best, again reaching 22.6% combined accuracy. Smaller values keep intervals short and over-schedule examples that have already been retained, reducing exposure for examples that genuinely need review. Larger values inflate intervals too quickly and let fragile examples drift out of the review schedule before they are stably retained. The default value chosen by the original SM-2 algorithm (Wozniak, 1998) is therefore a reasonable starting point for LLM continual pre-training as well.

E_0	Code Old	Code New	Combined
1.5	12.2 $\pm$ 1.5	14.2 $\pm$ 1.5	13.2 $\pm$ 1.1
2.5	24.4 $\pm$ 1.9	20.7 $\pm$ 1.8	22.6 $\pm$ 1.3
3.5	12.0 $\pm$ 1.4	9.0 $\pm$ 1.2	10.5 $\pm$ 1.0

Table 6: Sensitivity to the initial ease factor E_0 on TinyLlama code QA. The default SM-2 value $E_0 = 2.5$ achieves the best combined accuracy.

7 Discussion

The results support a specific claim: SRT improves continual pre-training when old and new knowledge must share a finite training budget and when retention difficulty is heterogeneous across examples. We discuss three implications of this finding.

Scheduling is the active ingredient, not exposure.

Naive continual pre-training is the headline practical comparison because it captures the realistic no-review update regime, but it does not isolate the mechanism behind SRT’s gains. Uniform replay serves this role. Because Uniform Replay and SRT share the same 20/80 exposure cap and pool structure, the only experimental variable separating them is whether within-pool selection is uniform or scheduled. On TinyLlama Wikipedia, this single variable is responsible for a 23.8 percentage-point gap in old-knowledge accuracy. The same ordering, $\text{CPT} < \text{UNIFORM REPLAY} < \text{SRT}$, holds on Llama-3.2-3B-Instruct Wikipedia. These results indicate that scheduling, not mere re-exposure, is what produces SRT’s retention advantage. Replay budgets that look adequate in aggregate can still fail when allocated uniformly across examples with heterogeneous retention difficulty.

Uniform replay can damage reasoning capabilities.

A more surprising finding is that uniform replay does not merely fall short of SRT on broad capabilities; it can be substantially worse than naive CPT itself. On Llama-3.2-3B-Instruct, Uniform

Replay drops BBH from a CPT baseline of 44.3% to 8.4% and GSM8K from 38.8% to 6.8%, while SRT preserves both close to the original model. We do not claim direct evidence for the underlying mechanism; one possible explanation is that re-exposing the model to old data without scheduling introduces interference dynamics that disrupt reasoning-relevant computation more severely than no replay at all, but verifying this would require representational analysis beyond the scope of this work. Regardless of mechanism, the result has an immediate practical implication: practitioners considering replay as a simple add-on to continual updates should be cautious, because a poorly designed replay schedule can degrade capabilities the no-replay baseline preserves.

SRT is a scheduling layer, not a replacement.

SRT addresses a question that mixture-level replay does not: among reviewable examples, which ones should return now? It does so without modifying the architecture, the training objective, or the choice of replay budget. SRT can therefore be combined with learning-rate rewarming (Ibrahim et al., 2024), data-mixture selection (Chen et al., 2025), parameter-efficient tuning, or synthetic rehearsal (Huang et al., 2024). Its contribution is orthogonal to these methods, addressing the within-pool selection problem they leave unspecified. We see scheduling as a complementary layer that practitioners can adopt alongside existing CPT recipes rather than as an alternative to them. The SRT scheduler is also architecture-agnostic: auxiliary experiments on vision and tabular classification benchmarks (Appendix C) show that the same scheduling, with predicted-class confidence in place of perplexity as the recall signal, mitigates class-incremental forgetting more effectively than naive continual training and Elastic Weight Consolidation.

8 Conclusion

We introduced Spaced Repetition Training, a cognitively inspired continual pre-training method that schedules per-example review using SuperMemo-2 and perplexity-derived recall quality. The central insight is that continual pre-training is not solely a mixture problem but a scheduling problem: among reviewable examples, the model should revisit those it is currently struggling to retain, not those it has already learned. On temporally grounded Wikipedia and code QA benchmarks across two

model scales, SRT recovers old-knowledge accuracy that naive continual pre-training discards, while matching or exceeding new-knowledge acquisition. At the larger model scale, SRT preserves broad capability benchmark performance that both naive CPT and uniform replay degrade, in some cases substantially. Together these results suggest that adaptive review timing is a simple, architecture-agnostic, and effective mechanism for improving the stability-plasticity trade-off in continual pre-training, and that established principles from human memory research transfer productively to language model training.

Limitations

Our language experiments use TinyLlama-1.1B and Llama-3.2-3B-Instruct, both from the Llama family, and evaluation is restricted to English; behaviour at larger scales, across different model families, and in multilingual settings remains open, even though our cross-modality evidence demonstrates the effectiveness of SRT. The temporal QA benchmarks are constructed for this work and should be interpreted as in-domain measurements of retention and acquisition on our specific corpora rather than as community-standard tests. The broad-capability results in Table 2 are obtained from a single training run per condition, evaluated over multiple evaluation seeds; the reported variance therefore reflects evaluation noise rather than training-run variability. While the reasoning-benchmark degradation under uniform replay is consistent across two independent benchmarks (BBH and GSM8K) and is not exhibited by SRT under identical conditions, which makes a single-run artifact less likely, confirming its stability would require multiple independent training runs. We leave this to future work. Finally, the perplexity-to-quality mapping uses fixed thresholds that may need recalibration for substantially different model scales or domains, and SRT introduces a forward pass per reviewed example beyond the normal training pass; a learned quality function and empirical overhead measurement at larger scales are natural extensions for future work.

Ethics and Reproducibility

Continual updates can introduce factual errors, preserve outdated information, or reinforce unsafe code patterns; source-grounded multiple-choice accuracy is not deployment reliability. Replay as-

sumes access to historical examples, so buffers and SRT state must respect retention limits, deletion requests, and data-governance obligations.

Exact benchmark artifacts, code, logs, and QA items are proprietary and are not released. Replication is supported by the algorithm, recall-quality mapping, corpus criteria, hyperparameters, evaluation protocol, and reported results, which are sufficient to recreate comparable public-data pipelines. Benchmark generation used LLM assistance with manual checks (Appendix A.3).

References

- Stefan Aeberhard, Danny Coomans, and Olivier De Vel. Comparative analysis of statistical pattern recognition methods in high dimensional settings. *Pattern Recognition*, 27(8):1065–1077, 1994.
- Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 7432–7439, 2020.
- Nicholas J. Cepeda, Harold Pashler, Edward Vul, John T. Wixted, and Doug Rohrer. Distributed practice in verbal recall tasks: A review and quantitative synthesis. *Psychological Bulletin*, 132(3):354–380, 2006.
- Jie Chen, Zhipeng Chen, Jiapeng Wang, Kun Zhou, Yutao Zhu, Jinhao Jiang, Yingqian Min, Wayne Xin Zhao, Zhicheng Dou, Jiaxin Mao, Yankai Lin, Ruihua Song, Jun Xu, Xu Chen, Rui Yan, Zhewei Wei, Di Hu, Wenbing Huang, and Ji-Rong Wen. Towards effective and efficient continual pre-training of large language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5779–5795, Vienna, Austria, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.acl-long.289.
- Jeffrey Cheng, Marc Marone, Orion Weller, Dawn Lawrie, Daniel Khashabi, and Benjamin Van Durme. Dated data: Tracing knowledge cutoffs in large language models, 2024.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. In *arXiv preprint arXiv:2110.14168*, 2021.
- Hermann Ebbinghaus. *Memory: A Contribution to Experimental Psychology*. 1885. English translation reprinted in *Annals of Neurosciences*, 2013.
- Yujie Feng, Hao Wang, Jian Li, Xu Chu, Zhaolu Kang, Yiran Liu, Yasha Wang, Philip S. Yu, and Xiao-Ming Wu. Forever: Forgetting curve-inspired memory replay for language model continual learning, 2026.
- Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation, 12 2023. URL <https://zenodo.org/records/10256836>.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A. Smith. Don’t stop pretraining: Adapt language models to domains and tasks. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8342–8360. Association for Computational Linguistics, 2020.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *Proceedings of the International Conference on Learning Representations*, 2021.
- Jianheng Huang, Leyang Cui, Ante Wang, Chengyi Yang, Xinting Liao, Linfeng Song, Junfeng Yao, and Jinsong Su. Mitigating catastrophic forgetting in large language models with self-synthesized rehearsal. *arXiv preprint arXiv:2403.01244*, 2024.
- Ching-Yi Hung, Cheng-Hao Tu, Cheng-En Wu, Chien-Hung Chen, Yi-Ming Chan, and Chu-Song Chen. Compacting, picking and growing for unforgetting continual learning. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- Adam Ibrahim, Benjamin Thérien, Kshitij Gupta, Mats L. Richter, Quentin Anthony, Timothée Lesort, Eugene Belilovsky, and Irina Rish. Simple and scalable strategies to continually pre-train large language models. *Transactions on Machine Learning Research*, 2024.
- Joel Jang, Seonghyeon Ye, Changho Lee, Sohee Yang, Joongbo Shin, Janghoon Han, Gyeonghun Kim, and Minjoon Seo. Temporalwiki: A lifelong benchmark for training and evaluating ever-evolving language models. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 6237–6250, 2022.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

- Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- Yann LeCun. The MNIST database of handwritten digits, 1998. URL <http://yann.lecun.com/exdb/mnist/>.
- Chen-An Li and Hung-yi Lee. Examining forgetting in continual pre-training of aligned large language models. *arXiv preprint arXiv:2401.03129*, 2024.
- Jeffrey Li, Mohammadreza Armandpour, Seyed Iman Mirzadeh, Sachin Mehta, Vaishaal Shankar, Raviteja Vemulapalli, Samy Bengio, Oncel Tuzel, Mehrdad Farajtabar, Hadi Pouransari, and Fartash Faghri. TiC-LM: A web-scale benchmark for time-continual LLM pretraining. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 32231–32273, Vienna, Austria, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.acl-long.1551.
- Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. *Psychology of Learning and Motivation*, 24:109–165, 1989.
- Meta AI. Llama 3.2: Revolutionizing Edge AI and Vision with Open, Customizable Models, September 2024. URL <https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/>.
- Meryem M’hamdi and Jonathan May. Leitner-guided memory replay for cross-lingual continual learning. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 7808–7821, Mexico City, Mexico, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.naacl-long.432.
- German I. Parisi, Ronald Kemker, Jose L. Part, Christopher Kanan, and Stefan Wermter. Continual lifelong learning with neural networks: A review. *Neural Networks*, 113:54–71, 2019.
- David Rolnick, Arun Ahuja, Jonathan Schwarz, Timothy Lillicrap, and Gregory Wayne. Experience replay for continual learning. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- Andrei A. Rusu, Neil C. Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks. In *arXiv preprint arXiv:1606.04671*, 2016.
- Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed H. Chi, Denny Zhou, and Jason Wei. Challenging big-bench tasks and whether chain-of-thought can solve them. *Findings of the Association for Computational Linguistics: ACL 2023*, pages 13003–13051, 2023.
- Piotr A. Wozniak. Supermemo 2 algorithm. Technical note, 1998.
- Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.
- Friedemann Zenke, Ben Poole, and Surya Ganguli. Continual learning through synaptic intelligence. In *International Conference on Machine Learning*, pages 3987–3995. PMLR, 2017.
- Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. Tinyllama: An open-source small language model, 2024.

A Corpora Construction Details

This appendix provides extended construction details for the Wikipedia and code corpora described in Section 4.1, together with the question-answering evaluation sets derived from each domain.

A.1 Wikipedia Corpora

Snapshot selection. We construct Wikipedia corpora from timestamped English Wikipedia snapshots. For TinyLlama-1.1B-Chat experiments, snapshot dates are chosen to bracket the model’s pre-training data assembly window in mid-2023, with the old snapshot near that window and the new snapshot drawn from a sufficiently later date that revisions reflect post-cutoff information. For Llama-3.2-3B-Instruct experiments, we use November 2023 and January 2024 snapshots. The short two-month gap creates a challenging setting in which the model must integrate recent factual updates while retaining closely related prior knowledge.

Diff extraction. For each aligned article pair, we segment both versions into sentences and compute sequence-level diffs to identify inserted, deleted, and modified spans. The $\mathcal{D}_{\text{new}}$ training corpus consists of inserted sentences together with the updated side of modified spans. This procedure reduces unchanged text while preserving localized factual updates relevant for continual pre-training, and ensures that $\mathcal{D}_{\text{new}}$ training examples reflect content not present in the corresponding $\mathcal{D}_{\text{old}}$ articles.

Topic stratification. For Llama-3.2-3B-Instruct experiments, candidate articles are stratified by topic and magnitude of change. Topics span politics, current affairs, computer science and artificial

Corpus	Construction	Rows	Old median words	New median words	Median sent. diff
TinyLlama reference diff	Added/modified update text	31,729	–	–	–
Llama-3 title-aligned reconstruction	2023-11 → 2024-01	28,946	543	546	0
Llama-3 topic-stratified corpus	2023-11 → 2024-01, topic/change stratified	31,729	593	614	3

Table 7: Wikipedia corpora used for language-model continual pre-training.

Domain	$\mathcal{D}_{\text{old}}$ Train	$\mathcal{D}_{\text{new}}$ Train	$\mathcal{D}_{\text{old}}$ QA	$\mathcal{D}_{\text{new}}$ QA
Wikipedia text	31,729	31,729	500	500
Source code	22,055	70,643	500	500

Table 8: Continual pre-training corpus sizes. Old examples represent pre-cutoff knowledge to be retained, and new examples represent post-cutoff information to be acquired. QA sets are held out exclusively for evaluation.

intelligence, business and economics, science and medicine, law and policy, and sports and entertainment. Stratification ensures balanced coverage across domains rather than over-representation of any single topic area.

A.2 Code Corpora

Repository selection. We collect repositories from GitHub using creation and commit timestamps as the temporal axis. Old examples come from repositories that existed before January 2022, while new examples come from repositories created after July 2024. We explicitly verify that repositories selected for $\mathcal{D}_{\text{new}}$ did not exist before the cutoff window to prevent temporal leakage between splits.

Language coverage. Repositories span thirteen widely used programming languages: Python, JavaScript, TypeScript, Java, C, C++, Go, Rust, C#, Ruby, PHP, Swift, Kotlin, and Scala. Each training example contains structured code text including the source file path and file content. Unlike the Wikipedia corpus, old and new code examples are not aligned revisions of the same artifact, reflecting that real software ecosystems evolve through new repositories and libraries rather than incremental revisions to existing files.

A.3 Evaluation Question-Answering Sets

For each domain, we construct held-out multiple-choice question-answering benchmarks with 500 questions per split, disjoint from the training corpora. Wikipedia questions are generated from held-out articles in an MMLU-style format (Hendrycks et al., 2021), with four answer choices per question and a single correct answer grounded in the source passage. Code-domain questions are generated analogously from held-out repositories and target repository-specific functionality, API usage, configuration semantics, and implementation details.

All questions are manually checked for answerability and source support. The $\mathcal{D}_{\text{old}}$ QA sets measure retention of pre-cutoff knowledge, while the $\mathcal{D}_{\text{new}}$ QA sets measure acquisition of post-cutoff information.

B Training Details

This appendix provides the full training configuration used in all continual pre-training experiments. The same hyperparameters are used for both TinyLlama-1.1B-Chat and Llama-3.2-3B-Instruct, and across all conditions (CPT, Uniform Replay, and SRT), so that performance differences reflect the replay strategy rather than optimization differences. Hyperparameters are summarized in Table 9.

Hardware and training duration. All experiments are conducted using bfloat16 mixed-precision training with gradient checkpointing enabled to reduce memory consumption. The effective batch size of 512 (per-device batch size $32 \times$ gradient accumulation 16) is held constant across model scales. Training is run for sufficient steps to consume a single pass over $\mathcal{D}_{\text{new}}$ at the configured replay ratio; exact step counts depend on corpus size and are listed in the experiment logs released with the code.

SRT thresholds. The perplexity thresholds in Table 9 correspond to the τ_k values defined in Eq. 7. Lower perplexity yields higher recall quality, with $q = 5$ assigned when $\text{PPL} < \tau_5 = 50$ and $q = 0$ assigned when $\text{PPL} \geq \tau_1 = 5000$. Sensitivity to a uniform scaling of these thresholds is analysed in Section 6.

Hyperparameter	Value
<i>Optimization</i>	
Optimizer	AdamW (fused)
β_1, β_2	0.9, 0.999
ϵ	$1e-8$
Weight decay	0.001
Gradient clipping (max norm)	1.0
<i>Learning rate schedule</i>	
Learning rate	$1e-4$
Scheduler	Linear
Warmup steps	200
<i>Batch and sequence</i>	
Per-device batch size	32
Gradient accumulation steps	16
Effective batch size	512
Sequence length	2,048
<i>Precision and memory</i>	
Mixed precision	bfloat16
Gradient checkpointing	Yes
<i>SRT-specific</i>	
Old-exposure ratio ρ	0.2
Initial ease factor E_0	2.5
Initial repetitions n_0	0
Initial interval I_0	1
Minimum ease factor	1.3
<i>Evaluation</i>	
QA accuracy bootstrap resamples	10,000

Table 9: Training and evaluation hyperparameters. The same configuration is used across all model scales and all replay conditions.

C Auxiliary Vision and Tabular Experiments

Although the central contribution of this paper concerns language model continual pre-training, the underlying SRT scheduler is architecture-agnostic and can be applied to other modalities. In an earlier version of this work, we evaluated SRT on small vision and tabular classification benchmarks under class-incremental learning, providing a sanity check that the scheduling mechanism generalizes beyond text. We summarize these auxiliary experiments here. The results are consistent with the main language-model findings: naive continual training rapidly forgets old classes, regularization-based methods provide partial mitigation, and SRT retains substantially more old-class accuracy while continuing to learn new classes.

C.1 Datasets and Class-Incremental Setup

We use four standard publicly available benchmarks spanning vision and tabular modalities: MNIST handwritten digits (LeCun, 1998), Fashion-MNIST clothing images (Xiao et al., 2017),

Dataset	Class assignment
MNIST	Old: $\{0, 1, 2, 3, 4, 5, 6\}$ New: $\{7, 8, 9\}$
Fashion	Old: T-shirt, Trouser, Dress, Coat, Sandal, Bag, Boot New: Pullover, Shirt, Sneaker
CIFAR-10	Old: Airplane, Automobile, Bird, Cat, Deer, Ship New: Truck, Frog, Dog, Horse
Wine	Old: classes $\{1, 2\}$ New: class $\{3\}$

Table 10: Class-incremental splits used in the auxiliary vision and tabular experiments. Vision datasets use semantically confusable class assignments to induce stronger interference.

CIFAR-10 natural images (Krizhevsky et al., 2009), and the UCI Wine dataset (Aeberhard et al., 1994). For each dataset, classes are partitioned into an *old* subset (used to train the base model) and a *new* subset (introduced sequentially in the continual update phase). For the image datasets, the partition uses semantically confusable classes to induce stronger interference between old and new representations. Table 10 reports the exact class assignments.

C.2 Recall Quality for Non-Generative Models

For language models, SRT derives the SuperMemo-2 recall quality from per-example perplexity. For classification models, perplexity is not defined, so we use the model’s predicted-class confidence in place of perplexity. Specifically, for an example x_i with true label y_i , the recall quality is computed from the predicted probability $p_\theta(y_i | x_i)$: high confidence in the correct label corresponds to high recall quality (longer next interval), while low confidence corresponds to low recall quality (shorter next interval). The remaining SM-2 update logic is identical to the language-model formulation in Section 3.2.

C.3 Baseline: Elastic Weight Consolidation

For the vision and tabular experiments, we additionally compare against Elastic Weight Consolidation (EWC) (Kirkpatrick et al., 2017), a regularization-based continual learning method that penalizes updates to parameters important for previously learned tasks. EWC estimates per-parameter importance using the diagonal of the Fisher Information Matrix computed on the old data and adds a

quadratic penalty

$$\mathcal{L}_{\text{EWC}}(\theta) = \mathcal{L}_{\text{new}}(\theta) + \frac{\lambda}{2} \sum_i F_i (\theta_i - \theta_i^*)^2, \quad (11)$$

where θ^* denotes the parameters at the end of training on the old data, F_i is the Fisher importance of parameter θ_i , and λ controls the regularization strength. EWC is a natural comparison for the vision setting because the small fully-connected and convolutional models used here permit Fisher computation at low overhead. In language-model continual pre-training, gradient-level regularization of this kind is significantly more expensive and is not the focus of our main experiments.

C.4 Model Architectures

All vision and tabular models are simple feed-forward or convolutional networks implemented in PyTorch with ReLU activations. The MNIST and Fashion-MNIST models use two convolution-pooling blocks followed by two fully connected layers. The CIFAR-10 model uses the same convolutional structure with dropout in the classifier head. The Wine model is a three-layer multilayer perceptron operating on the 13-dimensional UCI feature vector. All models are trained with cross-entropy loss and AdamW.

C.5 Results

Table 11 reports accuracy across all four datasets for the Base model, naive CPT, EWC, and SRT, averaged over 10 random seeds. The pattern is consistent across modalities. Naive CPT loses essentially all old-class accuracy on vision datasets while reaching high new-class accuracy, mirroring the catastrophic-forgetting failure mode observed at language scale. EWC partially mitigates forgetting on MNIST and Wine but performs poorly on Fashion-MNIST and CIFAR-10, where semantic similarity between old and new classes makes parameter-importance estimates less informative. SRT retains substantially more old-class accuracy than both baselines across all four datasets while maintaining competitive or superior new-class accuracy, achieving the highest overall accuracy in every setting.

These auxiliary experiments are not intended as central evidence for the central claim of this paper, which concerns language model continual pre-training. They do, however, suggest that the scheduling mechanism underlying SRT generalizes beyond text to non-generative classification

settings, where the recall quality signal is derived from predicted-class confidence rather than perplexity.

D External Validation on TemporalWiki

To provide external corroboration beyond our source-grounded QA, we evaluate on TemporalWiki (Jang et al., 2022), an established benchmark that measures factual knowledge via perplexity on subject-relation-object probes. Probes are categorised as *Changed* (facts that differ between consecutive Wikipedia snapshots) and *Unchanged* (facts that persist). Lower perplexity indicates better retention of the probed fact.

We score each probe by the perplexity the model assigns to the object tokens conditioned on the subject and relation, and report the mean over each category. Table 12 reports results for Base, CPT, and SRT on Llama-3.2-3B-Instruct.

CPT nearly triples perplexity on both categories relative to the base model, a clear signature of catastrophic forgetting. SRT limits the degradation to roughly 30% above base (2785 vs. 2112 average), corroborating on an external benchmark the retention advantage observed on our source-grounded QA.

Two caveats apply to this evaluation. First, absolute perplexity values are not comparable to those in the original TemporalWiki paper, which used GPT-2-family models; perplexity is tokenizer-dependent, so only the relative ordering across our conditions is meaningful. Second, the publicly released TemporalWiki snapshots predate the knowledge cutoff of Llama-3.2-3B-Instruct, so both the *Changed* and *Unchanged* categories represent pre-cutoff knowledge for our models. This evaluation therefore serves as an external *retention* check rather than a test of new-knowledge acquisition; constructing date-aligned probes that postdate the model cutoff is left to future work.

Dataset	Method	Old	New	Overall
MNIST	BASE	98.9 ± 0.6	0.0 ± 0.0	69.5 ± 0.4
	CPT	0.0 ± 0.0	98.3 ± 0.7	29.2 ± 0.2
	EWC	67.3 ± 5.4	57.3 ± 7.1	29.5 ± 0.2
	SRT	98.5 ± 1.1	75.1 ± 9.4	91.6 ± 2.6
Fashion	BASE	97.0 ± 0.4	0.0 ± 0.0	67.9 ± 0.0
	CPT	0.0 ± 0.1	94.4 ± 0.5	28.4 ± 0.2
	EWC	29.1 ± 0.2	95.3 ± 0.2	28.6 ± 0.0
	SRT	53.7 ± 4.3	94.2 ± 0.5	65.8 ± 3.1
CIFAR-10	BASE	83.6 ± 0.3	0.0 ± 0.0	50.1 ± 0.0
	CPT	0.0 ± 0.1	43.7 ± 27.2	17.5 ± 10.9
	EWC	20.4 ± 12.8	57.7 ± 31.2	23.1 ± 12.5
	SRT	43.5 ± 10.7	67.6 ± 21.6	53.1 ± 14.0
Wine	BASE	97.1 ± 0.2	0.0 ± 0.0	70.2 ± 0.0
	CPT	4.1 ± 0.1	100.0 ± 0.0	30.0 ± 0.0
	EWC	8.2 ± 0.2	100.0 ± 0.0	33.9 ± 0.1
	SRT	42.1 ± 0.1	82.7 ± 0.2	53.3 ± 0.1

Table 11: Vision and tabular results under class-incremental learning. Accuracy is reported as mean $\pm$ standard deviation over 10 random seeds (%). SRT achieves the highest overall accuracy on all four datasets.

Model	Changed PPL	Unchanged PPL	Avg. PPL
Base	1961	2263	2112
CPT	5713	6706	6209
SRT	2519	3052	2785

Table 12: TemporalWiki (Jang et al., 2022) object-perplexity on Llama-3.2-3B-Instruct (lower is better). CPT nearly triples perplexity relative to Base on both categories, indicating catastrophic forgetting, while SRT limits the degradation to roughly 30% above Base.